Relatório da atividade CI/CD - Revista Idibi

Data: 10/11/2026

Status: workflows separados implementados localmente. Execução no GitHub e publicação pública pendentes.

Projeto e estratégia

Repositório: https://github.com/JonesPrado/revista-idibi

Branch de trabalho: `apresentacao`. A branch de destino exigida é `main`.

O projeto é um site estático servido a partir de `index.html`, CSS, JavaScript e `assets/`. O README informa que não há etapa de compilação nem dependências npm para servir o site. GitHub Pages é uma estratégia adequada porque publica os arquivos estáticos e é citada no enunciado do laboratório como opção para esse tipo de projeto.

O workflow `.github/workflows/ci.yaml` executa CI em `push` para `main` e em `pull_request` destinado a `main`. O arquivo separado `.github/workflows/cd.yaml` usa `workflow_run` para iniciar depois da conclusão da CI e só continua quando a execução de origem foi bem-sucedida, veio de um `push` e pertence à branch `main`. O CD faz checkout do mesmo commit validado, prepara apenas os arquivos do site e publica pelo GitHub Pages. O job de publicação depende do job que envia o artefato.

Validações implementadas

- `ci.yaml` confere os arquivos principais do site com comandos inline e usa `node --check` para validar `figma-reader.js` e `page-narration.js`.

- `cd.yaml` monta `_site` com `index.html`, `styles.css`, os dois arquivos JavaScript e `assets/`, além de `.nojekyll`; não publica os arquivos auxiliares do repositório.

- Não foi adicionado script auxiliar, instalação npm, Pillow ou etapa de compilação. Os scripts já existentes no projeto não foram alterados nem são chamados pelos workflows.

Alterações realizadas

- Separado o conteúdo em `.github/workflows/ci.yaml` para CI e `.github/workflows/cd.yaml` para CD. O arquivo `cd.yaml`, que havia sido removido vazio na primeira versão, foi recriado com o gatilho `workflow_run` e a condição de sucesso para `push` em `main`.

- Removido `.github/scripts/validate_site.py` conforme a correção do escopo; as verificações ficaram inline em `ci.yaml`.

- Atualizado este registro e o relatório PDF, incluindo três molduras vazias para os prints solicitados.

- Nenhum arquivo da aplicação foi alterado. A pasta `tests/` apareceu como não rastreada na inspeção inicial e continha testes que referenciavam `src/`, ausente neste repositório. Ela não foi alterada pelo agente nem incluída na simulação do artifact; na inspeção final, já não estava no workspace. Não foi possível determinar a origem dessa mudança.

- Nenhuma edição manual do usuário foi observada durante esta implementação. Não foi feito commit, push, merge, alteração de configuração externa ou envio do formulário.

Verificações locais e evidências

- `node --check figma-reader.js` e `node --check page-narration.js` - aprovados localmente em Node.js v24.19.0.

- Os dois arquivos YAML foram analisados com PyYAML 6.0.3. Foram conferidos os gatilhos e as condições: CI em `push`/`pull_request` para `main`; CD após `workflow_run` concluído com sucesso, somente para o evento original `push` em `main`; deploy depende do job de preparação e usa permissões Pages/OIDC.

- A montagem do diretório de publicação foi simulada localmente com os arquivos estáticos selecionados; os arquivos auxiliares não entram no artifact.

O workflow ainda não foi executado no GitHub porque as alterações permanecem na branch local `apresentacao` e não houve push ou merge. Portanto, não há URL de execução nem evidência de deployment remoto nesta versão do relatório. A publicação e a disponibilidade do site continuam pendentes.

Antes da publicação, uma pessoa com acesso de maintainer/admin deve abrir **Settings > Pages** no repositório e selecionar **GitHub Actions** como origem de Build and deployment. Depois, deve abrir um pull request de `apresentacao` para `main`, conferir a CI e, após o merge, verificar o job de deployment na aba Actions e visitar o endereço fornecido pelo próprio job. O endereço esperado para o site do projeto é `https://jonesprado.github.io/revista-idibi/`, ainda não confirmado.

Documentação oficial consultada:

- https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages

- https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

- https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#workflow_run

Registro de prompts e evolução

Foram 4 prompts relevantes neste histórico: 2 de planejamento e 2 durante a implementação (pedido inicial e correção). Resultados remotos não foram inventados.

Prompt 1 - planejamento

```
Vamos fazer um plano de implementação primeiro, veja o que teremos de fazer
```

Resultado: leitura do enunciado e inspeção inicial do repositório; foi proposto um plano para CI em `main`, CD com GitHub Pages e registro dos prompts. Nenhum arquivo foi alterado nessa etapa.

Prompt 2 - preparar o prompt de implementação

```
Monte o prompt para fazer tudo isso
```

Resultado: foi preparado um prompt de implementação com os requisitos do PDF, preservação de alterações existentes, verificações locais, relatório e proibição de afirmar evidências remotas não observadas. Nenhum arquivo foi alterado nessa etapa.

Prompt 3 - implementação

```
Quero que você implemente a atividade de CI/CD descrita no PDF:

C:/Users/joaoprado-ieg/OneDrive - Instituto J&F/Área de Trabalho/Tech/2026/Desenvolvimento Operações Ágeis/CI
CD/19b - Lab CICD-Pipeline.pdf

Trabalhe no repositório aberto em C:\~Idibi\idibi-html. O remoto é

https://github.com/JonesPrado/revista-idibi.

Antes de alterar qualquer arquivo:

1. Leia o PDF e extraia os requisitos da atividade.

2. Inspecione o estado do Git, a branch atual, os arquivos e a configuração do projeto.

3. Preserve alterações existentes. Em particular, verifique os arquivos vazios e não rastreados

em .github/workflows/ci.yaml e .github/workflows/cd.yaml antes de decidir como incorporá-los.

4. Apresente um resumo curto do que encontrou e da abordagem que pretende implementar.

Contexto conhecido do projeto:

- É um site estático composto por HTML, CSS, JavaScript e assets.

- O README informa que o site não precisa de npm nem de uma etapa de compilação para ser servido.

- Existem scripts Python de geração de gráficos e narração. O de gráficos usa Pillow; o de narração depende do
OCR do Windows.

- A branch atualmente aberta é apresentacao. O repositório tem uma branch main, que é o destino exigido pela
atividade.

Objetivo:

Implementar uma pipeline GitHub Actions adequada ao site, com CI em cada push para main e em pull requests
direcionados a main, e deployment automático após push para main somente quando a CI passar. Use GitHub Pages,
salvo se a inspeção revelar que o projeto já usa outro serviço compatível e automatizado.

Requisitos de implementação:
```

```
- Faça checkout do repositório.

- Configure somente o ambiente necessário para validar o site.

- Não adicione npm, dependências ou build desnecessários ao site estático.

- Inclua verificações automatizadas úteis para este projeto, como sintaxe dos arquivos JavaScript e uma
verificação de que os assets locais referenciados pelo site existem.

- Se propuser verificar a geração dos gráficos, instale e fixe a dependência necessária e explique por que
isso é útil. Não execute o script de OCR do Windows em um runner Linux.

- Faça o deploy apenas em push para main e faça o job de deploy depender explicitamente do job de validação,
por exemplo usando needs.

- Use GitHub Pages com as permissões e actions apropriadas, consultando a documentação oficial atual se
estiver disponível.

- Evite workflows duplicados. Prefira um workflow que contenha os jobs de CI e CD com a dependência explícita
entre eles. Resolva os arquivos vazios existentes com cuidado e explique qualquer remoção ou renomeação.

- Não coloque segredos ou tokens diretamente no código.

- Não faça commit, push, merge, alteração de configurações externas ou envio do formulário de entrega sem
minha autorização explícita.

Registro e entrega:

- Crie um registro em português dos prompts usados durante a implementação, dos resultados relevantes, dos
problemas encontrados, das correções e das alterações manuais.

- Conte os prompts usados na implementação. Não invente interações anteriores nem resultados.

- Prepare um relatório final em PDF com o link do repositório, o arquivo da pipeline, os prompts, o histórico
de alterações, uma análise breve e as evidências reais da execução.

- Se a pipeline ainda não tiver sido executada em GitHub ou se o site ainda não tiver sido publicado, deixe
isso claramente marcado como pendente. Não crie capturas de tela, links de execução ou evidências fictícias.

- Não envie o relatório pelo formulário do curso.

Verificação e conclusão:

- Execute as verificações locais pertinentes à implementação e informe os comandos e resultados.

- Confirme por inspeção que o gatilho contempla push para main e pull_request para main, e que o deployment
depende de CI bem-sucedida e só ocorre no push para main.

- Se não puder conferir a configuração do GitHub Pages ou a publicação, indique exatamente o passo manual
necessário para concluir essa parte.

- Ao terminar, resuma os arquivos alterados, o que foi verificado, o que falta para a publicação ou entrega e
onde está o PDF preparado.
```

Resultado: a primeira versão usou um workflow único e um script auxiliar de validação. O pedido de correção seguinte especificou dois arquivos YAML separados, sem script auxiliar, e áreas reservadas para três prints. Esses pontos foram ajustados. Execução remota e deployment continuam pendentes. Uma primeira simulação de staging tentou escrever na pasta temporária do sistema e recebeu acesso negado; ela foi refeita no workspace e passou. O Python padrão não tinha ReportLab; o PDF foi gerado com o runtime Python empacotado do workspace. A primeira geração do relatório falhou por conflito de formatação em URLs dentro de trechos de código; removi a formatação automática e gerei o PDF com sucesso. Uma checagem local inicial de arquivos usou incorretamente o parâmetro `Test-Path` repetido no PowerShell; corrigi o comando e a verificação passou.

Prompt 4 - correção da implementação

```
Tinha que ter um ci.yaml, um cd.yaml, e nada disso de scripts. além disso, voce deve deixar espaço pra eu
colocar 3 prints, mostrando a tela da pipeline, a tela de se a pipeline rodou, e mais uma
```

Resultado: separei CI e CD em `ci.yaml` e `cd.yaml`, removi o script auxiliar e acrescentei espaços de imagem no relatório. O terceiro espaço está identificado como captura da aplicação publicada no GitHub Pages.

Análise breve

A pipeline mantém os arquivos CI e CD separados. O `workflow_run` faz o CD começar após a conclusão da CI e a condição do job restringe o deploy a uma CI bem-sucedida originada por push em `main`. A atividade ainda não tem comprovação ponta a ponta porque não houve execução no GitHub, configuração da origem Pages nem publicação após merge em `main`.

Pipeline: ci.yaml

Conteudo lido diretamente de .github/workflows/ci.yaml.

```
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  validate:
    name: Validar site estático
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do repositório
        uses: actions/checkout@v6

      - name: Configurar Node.js
        uses: actions/setup-node@v7
        with:
          node-version: '24'

      - name: Conferir arquivos do site
        run: |
          test -f index.html
          test -f styles.css
          test -f figma-reader.js
          test -f page-narration.js
          test -d assets

      - name: Verificar sintaxe JavaScript
        run: |
          node --check figma-reader.js
          node --check page-narration.js
```

Pipeline: cd.yaml

Conteudo lido diretamente de .github/workflows/cd.yaml.

```yaml
name: CD

on:
  workflow_run:
    workflows:
      - CI
    types:
      - completed
    branches:
      - main

permissions:
  contents: read

concurrency:
  group: github-pages
  cancel-in-progress: false

jobs:
  prepare:
    name: Preparar site para publicação
    if: >-
      ${{
        github.event.workflow_run.conclusion == 'success' &&
        github.event.workflow_run.event == 'push' &&
        github.event.workflow_run.head_branch == 'main'
      }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do commit validado pela CI
        uses: actions/checkout@v6
        with:
          ref: ${{ github.event.workflow_run.head_sha }}
          persist-credentials: false

      - name: Configurar GitHub Pages
        uses: actions/configure-pages@v5

      - name: Montar diretório do site
        run: |
          mkdir _site
          cp index.html styles.css figma-reader.js page-narration.js _site/
          cp -R assets _site/assets
          touch _site/.nojekyll

      - name: Enviar artefato do GitHub Pages
        uses: actions/upload-pages-artifact@v4
        with:
          path: _site

  deploy:
    name: Publicar no GitHub Pages
    needs: prepare
    if: >-
      ${{
        github.event.workflow_run.conclusion == 'success' &&
        github.event.workflow_run.event == 'push' &&
        github.event.workflow_run.head_branch == 'main'
      }}
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - name: Publicar artefato validado
        id: deployment
        uses: actions/deploy-pages@v4
```

Evidencias visuais - inserir prints

Os espacos abaixo foram reservados para as capturas da execucao real.

Print 1 - Tela dos workflows CI e CD

Cole aqui a captura mostrando ci.yaml e cd.yaml na pasta .github/workflows.

Print 2 - Execucao da pipeline

Cole aqui a captura da execucao no GitHub Actions, com o resultado da CI e do CD.

Print 3 - Aplicacao publicada

Interpretado como a tela do site publicado no GitHub Pages, com a URL visivel.

Cole aqui a captura do site publicado. Este espaco ainda esta vazio; a publicacao precisa ser verificada.